S02

Ordenar y representar datos ecológicos

PROGRAMACIÓN EN R

Los datos de campo casi nunca llegan listos para analizar. Hoy tomas tres meses de censos de aves, corriges sus problemas, los pasas a formato ordenado, los unes con la información de los puntos y de las especies, y los conviertes en tablas resumen y figuras de calidad.

DURACIÓN	Unas 2 horas, con una pausa.
NECESITAS	R y RStudio con el `tidyverse` instalado, y la carpeta del proyecto de la semana 1.
ARCHIVOS	`semana02_ordenar_y_graficar.R`, `semana02_soluciones.R`, `bird_counts_wide.csv`, `species_traits.csv`, `landcover_classes.csv`, `survey_points.csv`
DATOS	Los 40 puntos de la semana 1, visitados en abril, mayo y junio de 2026; conteos de 10 especies de aves. Simulados.

Objetivos

Al acabar la sesión sabrás:

- encadenar pasos con la tubería `|>`;
- detectar y corregir texto inconsistente, fechas guardadas como texto y valores perdidos;
- pasar datos de formato ancho a largo y viceversa con `pivot_longer()` y `pivot_wider()`;
- unir tablas con `left_join()`;
- resumir por grupos para obtener la riqueza y la abundancia por punto y por hábitat;
- crear y guardar figuras con `ggplot2`.

1 · Datos ordenados (tidy)

Un conjunto de datos está **ordenado** cuando cada variable es una columna, cada observación una fila y cada valor una celda. Las planillas de campo suelen estar en formato **ancho**: una columna por especie, porque es rápido de rellenar. Para analizar las queremos en formato **largo**: una fila por punto, visita y especie.

Ancho, como se anotó en campo (primeras columnas):

point_id	visit	date	observer	european_robin	common_chaffinch	…
P01	1	14/04/2026	M. Puig	1	4	...
P01	2	12/05/2026	M. Puig	2	2	...

Largo, listo para dplyr y ggplot2:

point_id	visit	date	observer	species	count
P01	1	2026-04-14	M. Puig	european_robin	1
P01	1	2026-04-14	M. Puig	common_chaffinch	4
...					

La tubería (pipe)

`x |> f()` significa "coge x y luego aplícale f". Permite escribir un análisis como una secuencia de pasos que se lee de arriba abajo. Atajo: **Ctrl + Mayús + M** (**Cmd + Mayús + M** en Mac). En código antiguo verás %>%, que aquí funciona igual.

```
counts |>
  filter(visit == 1) |>          # solo la visita de abril
  select(point_id, woodlark) |>  # dos columnas
  arrange(desc(woodlark))        # primero los de más alondras
```

2 · Limpieza: mirar antes de analizar

El archivo de conteos tiene tres tipos de problema muy habituales en datos reales. Encontrarlos es parte del trabajo.

Problema	Cómo detectarlo	Solución en el script
Nombres de observadores escritos de siete formas ("M. Puig", "m. puig", "M.Puig"...)	`count(observer)`	Quitar espacios y puntos, pasar a mayúsculas y asignar a cada clave una grafía con `case_when()`
Fechas guardadas como texto, día/mes/año	`glimpse()` muestra <chr>	`dmy()` de lubridate las convierte en fechas
6 conteos perdidos (NA) en tres especies	`summarise(across(..., ~ sum(is.na(.))))`	Mantener NA distinto de 0 y quitar esas filas tras reorganizar

```
counts <- counts |>
  mutate(
    obs_key  = str_to_upper(str_remove_all(observer, "[ .]")),  # "MPUIG"
    observer = case_when(obs_key == "MPUIG"   ~ "M. Puig",
                         obs_key == "JSERRA"  ~ "J. Serra",
                         obs_key == "AFERRER" ~ "A. Ferrer")
  ) |>
  select(-obs_key) |>
  mutate(date = dmy(date))
```

Nunca corrijas los datos originales a mano

Es tentador corregir las erratas en Excel. Hazlo en el script: el archivo original queda intacto, cada cambio queda documentado y, si te mandan una versión actualizada, vuelves a ejecutar el script y obtienes la misma limpieza.

3 · Reorganizar y unir

```
counts_long <- counts |>
  pivot_longer(cols = european_robin:short_toed_treecreeper,
               names_to = "species", values_to = "count") |>
  filter(!is.na(count))        # 40 x 3 x 10 = 1.200 filas, menos 6 NA = 1.194

birds <- counts_long |>
  left_join(traits, by = "species") |>   # nombre científico, familia, dieta...
  left_join(points, by = "point_id")     # hábitat, altitud, coordenadas
```

`left_join()` conserva todas las filas de la tabla izquierda y añade las columnas que coinciden de la derecha. Comprueba siempre el número de filas antes y después de una unión: si crece, la clave (`species` o `point_id`) está repetida en la tabla de la derecha. En la versión en castellano del script también unimos `landcover_classes.csv` para tener los nombres de los hábitats en castellano en los gráficos.

4 · Riqueza y abundancia

Dos medidas clásicas de comunidad, calculadas por punto a partir de las tres visitas:

- **Riqueza de especies**: número de las 10 especies censadas vistas al menos una vez.
- **Abundancia**: total de aves contadas dividido entre el número de visitas (aves por visita).

Al resumir de nuevo por hábitat se obtiene esta tabla:

Hábitat	Puntos	Riqueza media	DE	Aves por visita
pinar de repoblación	5	8,8	1,1	9,6
encinar	9	8,4	0,7	11,7
bosque mixto	6	8,0	1,1	13,2
hayedo	9	7,0	1,2	11,4
brezal	6	5,3	1,4	5,2
prado	5	4,8	0,4	5,4

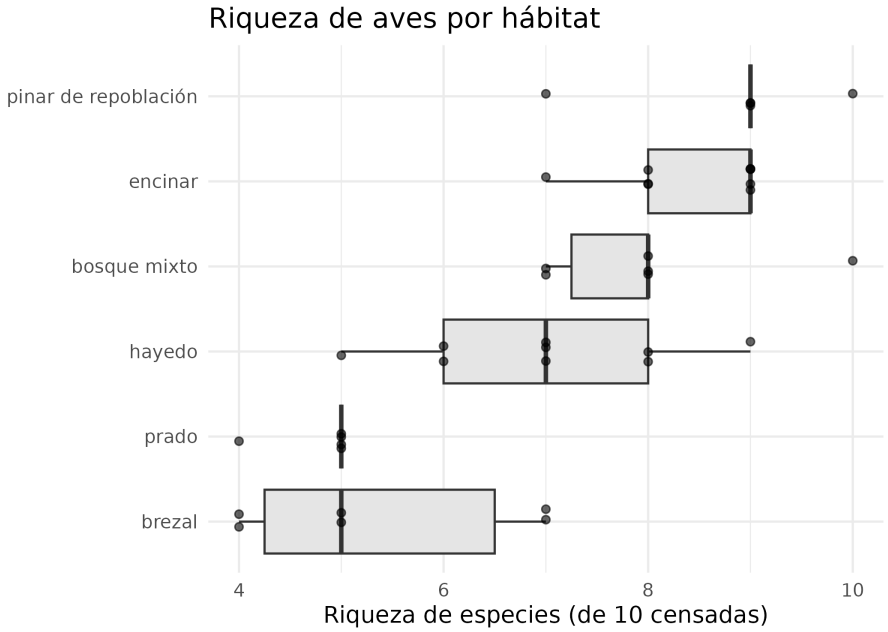
Piensa como ecólogo

Los medios abiertos tienen menos especies y menos aves que el bosque. Antes de creerlo, pregúntate: ¿hay suficientes puntos por hábitat (solo 5 en prado)? ¿Se detecta igual un ave en un prado abierto que en un hayedo cerrado? Los conteos brutos mezclan abundancia real y detectabilidad. Volvemos a ello con modelos en las semanas 3 y 4.

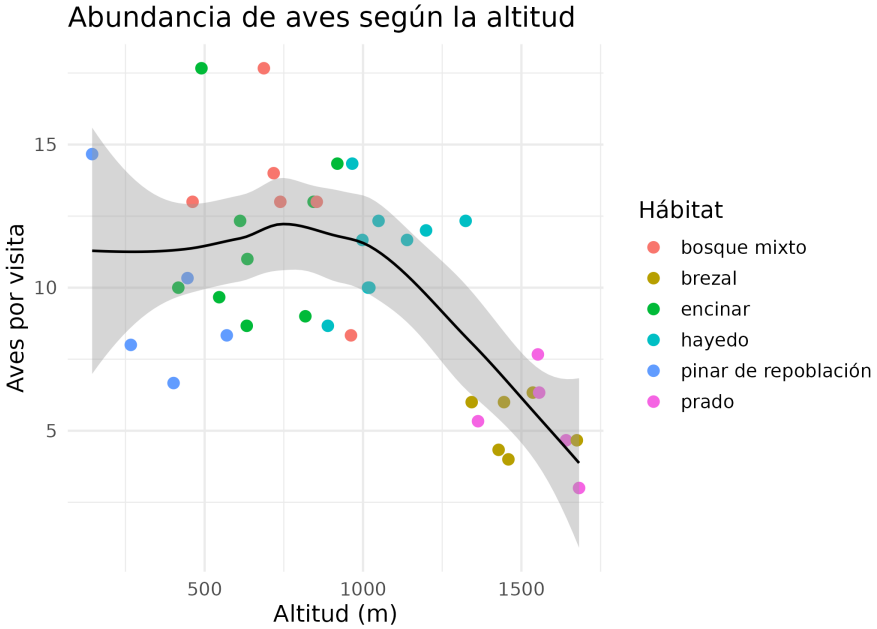
5 · ggplot2: la gramática de los gráficos

Todo ggplot se construye con las mismas piezas: los **datos**, las **estéticas** que asignan columnas a propiedades visuales (x, y, color, tamaño) y las **geometrías**, las marcas que se dibujan. Las capas se añaden con +.

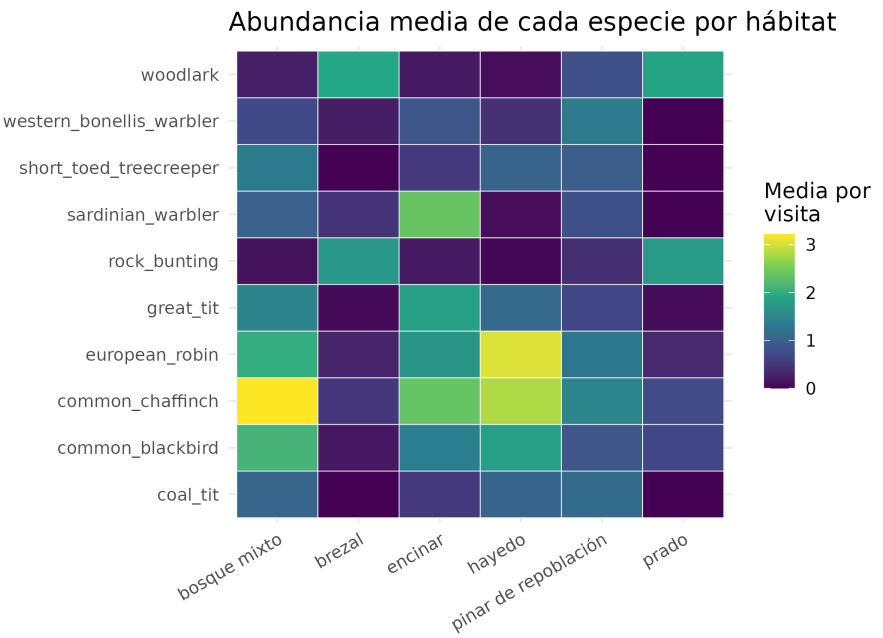
```
ggplot(point_summary, aes(x = elevation_m, y = abundance)) +   # datos + estéticas
  geom_point(aes(colour = habitat_es), size = 2.5) +           # capa 1: puntos
  geom_smooth(method = "loess", colour = "black") +            # capa 2: tendencia
  labs(x = "Altitud (m)", y = "Aves por visita") +              # ejes con unidades
  theme_minimal()
```



Riqueza por hábitat: diagrama de cajas con los puntos encima, para que se vea el tamaño de muestra.



Abundancia según la altitud, con un suavizado loess y su banda de confianza. La abundancia cae por encima de unos 1.300 m, donde el bosque da paso al medio abierto.



Abundancia media de cada especie por visita en cada hábitat. La alondra totovía y el escribano montesino dominan los medios abiertos; el pinzón y el petirrojo, los bosques.

Guarda las figuras desde el código con `ggsave()`, indicando el tamaño en pulgadas y 300 ppp, que aceptan casi todas las revistas. Así cada figura se puede recrear idéntica.

Funciones clave de hoy

Función	Qué hace
`read_csv(), write_csv()`	Leer y escribir CSV (readr)
`glimpse(), count()`	Ver columnas; contar filas por categoría
`filter(), select(), arrange()`	Quedarse con filas, con columnas, ordenar
`mutate()`	Crear o cambiar columnas
`group_by() + summarise()`	Una fila resumen por grupo
`pivot_longer(), pivot_wider()`	Cambiar entre formato ancho y largo
`left_join()`	Añadir columnas de otra tabla mediante una clave
`case_when()`	Asignar valores según condiciones
`dmy()`	Convertir texto día/mes/año en fechas (lubridate)
`ggplot() + geom_*()`	Construir un gráfico capa a capa
`ggsave()`	Guardar un gráfico en un archivo

Ejercicios

Al final del script. Soluciones en `semana02_soluciones.R`.

1. ¿Cuántas aves anotó cada observador? ¿Quién anotó más *por visita*, y por qué esa comparación es más justa?
2. Busca las tres especies con mayor conteo total.
3. Crea una columna que distinga bosque y medio abierto (`if_else()` y `%in%`) y compara la riqueza media.

4 Representa la cobertura de copas frente a la riqueza, con color por hábitat y una recta (`geom_smooth(method = "lm")`).

5 Gráfico de barras del conteo medio por dieta y hábitat (`geom_col(position = "dodge")`).

6 **Reto:** vuelve a pasar la tabla de especies por hábitat a formato ancho y guárdala.

Antes de la semana 3

- Termina los ejercicios y compáralos con las soluciones.
- Lectura opcional: capítulos de transformación de datos, datos ordenados y uniones de *R for Data Science*.